

Table des matières

Définitions des différentes notions vues en cours	2
I. Bases, Structure et Géométrie des Couches	2
II. Mécanismes d'Apprentissage et Régularisation	3
PROCESSUS D'APPRENTISSAGE D'UN RESEAU DE NEURONES	4
III. Fonctions d'Activation	5
IV. Évaluation et Métriques	5
V. Architectures et Tâches de Vision	6
VI. Architectures NLP (Natural Language Processing)	6
Utilisation des formules :	7
II. Fonctions d'Activation	7
2. ReLU (Rectified Linear Unit)	7
3. Sigmoid	7
4. SoftMax	7
5. Softplus	7
6. tanh (Tangente Hyperbolique)	7
III. Fonction de Perte (Loss)	7
7. Cross-Entropy/Log Loss (pour classification binaire)	7
IV. Métriques d'Évaluation	8
8. Precision (Précision)	8
9. Recall (Rappel)	8
10. F1 Score	8
Deux modèles appris en TD :	8
ALEXNET	8
Fonctionnement Détaillé de l'Architecture AlexNet (8 Couches)	9
CODE PYTHON ALEXNET	10
UNET	15
L'Architecture U-Net : Descriptif Détaillé	15
Programme UNET :	18
Types de questions à l'examen	23

Définitions des différentes notions vues en cours

I. Bases, Structure et Géométrie des Couches

Notion	Définition / Explication
Deep Learning	Champ d'étude qui utilise des architectures de réseaux de neurones.
CNN (Convolutional Neural Network)	Type de réseau utilisé pour traiter les données d'entrée. Son but est d'extraire les caractéristiques (<i>features</i>) du volume d'entrée.
Tensor	Structure de données essentielle dans le <i>Deep Learning</i> . Il peut être vu comme une matrice multi-dimensionnelle. Dans PyTorch, les tenseurs sont utilisés pour tout : valeurs d'entrée, poids, biais et résultats de sortie.
Neuron Unit	Composant de base du neurone, constitué d'un poids (<i>weight</i>), d'une fonction d'activation, et d'une valeur de sortie (<i>output value</i>).
Couche de Convolution	Couche dont le but est d'extraire les caractéristiques de l'entrée. Il est nécessaire de définir la taille de chaque filtre sur les couches.
Filtre / Kernel	Masque dont il faut définir la taille. Les cartes de caractéristiques (<i>Feature maps</i>) sont les sorties des filtres appliqués à l'image d'entrée.
Feature Maps	Représentent les activations de sortie des couches de convolution. Elles fournissent un aperçu de la manière dont le réseau traite et interprète les données d'entrée à diverses étapes.
Couche de Pooling (Pooling layer)	Composant standard utilisé dans les architectures CNN classiques pour l'extraction de caractéristiques. Couche qui réduit les dimensions spatiales de l'entrée. Elle permet de contrôler le surapprentissage (<i>overfitting</i>).
Max Pooling	Une des plusieurs fonctions disponibles pour la couche de <i>pooling</i> , avec l' <i>average pooling</i> et la norme L2. Unet utilise un MaxPool2D pour réduire les dimensions spatiales des images traitées tout en conservant les infos les plus importantes.
Couche Complètement Connectée (FC)	Couche qui utilise en entrée un vecteur où chaque donnée est connectée à un neurone. Son objectif principal est d'accomplir la tâche (classification, obtention de scores, etc.).
Stride	Nombre de pixels de la fenêtre glissante pour la convolution ou le <i>pooling</i> .
Zero-padding	Ajout de valeur pour ajuster la taille de l'image/matrice.
Équation de Taille de Couche	Formule pour ajuster la taille de chaque couche <u>O</u> de sortie, où <u>W</u> est la taille de l'entrée, <u>P</u> le <i>zero-padding</i> , <u>F</u> la taille du filtre et <u>S</u> la valeur du <i>stride</i> : $O = ((W - F + 2P)/S) + 1$.

Exemple :

Enoncé : Vous avez une image de taille 224X224 et un masque de convolution de 3X3, lorsque vous aurez appliqué ce masque sur toute l'image, quelle sera la taille de la matrice résultante ?

Méthodologie et Formule

Pour déterminer la taille de la matrice de sortie *O* (ou *Output size*), il est nécessaire de connaître la taille de l'entrée *W* (largeur ou hauteur), la taille du filtre (ou masque) *F*, la valeur du *Zero-padding* *P* et la valeur du *Stride* *S*.
L'équation à respecter pour ajuster la taille de chaque couche à partir de l'entrée est :

$$O = \frac{(W - F + 2P)}{S} + 1$$

Dans votre cas :

- *W*=224 (taille de l'image d'entrée).
- *F*=3 (taille du masque de convolution).

Puisque les valeurs de *P* (*Zero-padding*) et *S* (*Stride*) ne sont pas spécifiées, nous allons utiliser les valeurs par défaut souvent appliquées dans les architectures simples de CNN :

- *P*=0 (Pas de *Zero-padding*).
- *S*=1 (Pas de chevauchement de pixel, *stride* de 1).

Calcul

En utilisant la formule avec *W*=224, *F*=3, *P*=0 et *S*=1 : *O*=((224-3+2x0)/1)+1 = (221/1)+1 = 221+1=**222**

Réponse

La taille de la matrice résultante après l'application du masque de convolution 3×3 sur l'image 224×224, en supposant un *Stride* de 1 et un *Zero-padding* de 0, sera de 222×222.

II. Mécanismes d'Apprentissage et Régularisation

Notion	Définition / Explication
Époque (<i>Epoch</i>)	Définit le nombre de fois que l'algorithme d'apprentissage va parcourir l'ensemble du jeu de données d'entraînement. Le jeu d'entraînement est entièrement vu à chaque époque.
Propagation Avant (<i>Forward Propagation</i>)	Première étape du processus d'apprentissage où les données d'entraînement sont utilisées et la propagation avant est appliquée.
Rétropropagation (<i>Backpropagation</i>)	Algorithme d'apprentissage pour améliorer les sorties. Il corrige l'erreur en utilisant la fonction de perte. Il calcule le gradient de perte pour chaque coefficient.
Descente de Gradient (<i>Gradient Descent</i>)	Utilisée pour calculer l'évolution d'une fonction en choisissant la direction de la pente. Les poids sont ajustés pour exécuter la prochaine époque. Peut être complétée par l'utilisation d'un <i>momentum</i> (mémoire des gradients précédents) pour accélérer la convergence.
Chain Rule (Règle de la chaîne)	Règle appliquée lors de la rétropropagation : le gradient est calculé pour tous les poids et pour toutes les couches, de la dernière à la première.
Taux d'Apprentissage (<i>Learning Rate</i>)	Contrôle l'ampleur du changement du modèle en réponse à l'erreur estimée. S'il est trop petit, le temps de calcul peut être allongé ; s'il est trop large, la mise à jour peut être instable.
Batch Size (Taille de lot)	Définit le nombre d'échantillons à traiter avant de mettre à jour les paramètres internes du modèle.
Dropout Layer	Technique de régularisation pour prévenir le surapprentissage (<i>overfitting</i>) pendant l'entraînement. Elle retire aléatoirement une partie des neurones, typiquement entre 20% et 50%. (ALEXNET)
BatchNorm (Batch Normalization)	Vise à modifier les données du tenseur pour obtenir une valeur moyenne égale à 0 et une variance égale à 1. Ceci permet d'obtenir des données dans la même plage que la fonction d'activation.
Data Augmentation	Modification des données pour générer de nouveaux échantillons afin de réduire le surapprentissage. AlexNet a utilisé l'augmentation de données 2048 fois.
Transfert Learning	Réutilisation d'un modèle pré-entraîné pour alimenter un autre CNN pour une autre tâche ou un autre jeu de données. Seules les couches finales sont réentraînées, le reste du modèle étant gelé (<i>frozen</i>).
Fine-Tuning	Permet à certaines ou toutes les couches d'un modèle pré-entraîné d'être réentraînées (ajustées) sur un nouveau jeu de données.

1. Propagation Avant (Forward Propagation)

La Propagation Avant est la première phase du processus d'apprentissage.

Définition synthétique (à corriger)	Première étape du processus d'apprentissage où les données d'entraînement sont utilisées et la propagation avant est appliquée.
Rôle clarifié (basé sur les sources)	C'est l'étape où les données d'entraînement sont introduites dans le réseau. Les données traversent le modèle, de la première couche jusqu'à la dernière, où les transformations non-linéaires sont appliquées par les fonctions d'activation. À ce stade, le modèle produit une prédiction de sortie ($\hat{y}$).

Explication du principe :

- Saisie des données** : L'étape commence par la prise des données d'entraînement.
- Calcul** : Les données sont traitées couche par couche (couches de convolution, de *pooling*, complètement connectées). Les tenseurs, qui sont la structure de données essentielle dans le *Deep Learning*, subissent les opérations.
- Transformation** : Des fonctions d'activation (telles que ReLU, Sigmoid ou SoftMax) appliquent des transformations non-linéaires.
- Résultat** : Le réseau génère un résultat de sortie (la prédiction).

2. Rétropropagation (Backpropagation)

La Rétropropagation est la deuxième phase du processus d'apprentissage et un algorithme clé pour l'amélioration du modèle.

Définition synthétique (à corriger)	Algorithme d'apprentissage pour améliorer les sorties. Il corrige l'erreur en utilisant la fonction de perte. Il calcule le gradient de perte pour chaque coefficient.
Rôle clarifié (basé sur les sources)	C'est l'algorithme d'apprentissage utilisé pour améliorer les sorties en corrigeant l'erreur mesurée par la fonction de perte. Le but est d'obtenir le gradient de perte de chaque coefficient pour ensuite ajuster les poids du réseau.

Explication du principe :

- Mesure de l'Erreur** : Après la Propagation Avant, une **fonction de perte** (*Loss Function*) (comme la *Cross-Entropy*) est utilisée. Son but est de **minimiser le taux d'erreur** entre la prédiction et la vérité terrain (*groundtruth*). La fonction de perte estime le niveau d'erreur.
- Calcul du Gradient** : La rétropropagation utilise cette erreur (la perte) pour calculer le **gradient de perte** pour chaque poids (*coefficient*) du réseau.
- Application de la Règle de la Chaîne (*Chain Rule*)** : Pour calculer ce gradient, la Règle de la Chaîne est appliquée : le gradient est calculé pour tous les poids et pour toutes les couches, **de la dernière à la première**.
- Objectif** : Le but est de calculer l'impact de chaque paramètre et de le modifier en conséquence.
- Mise à jour des Poids** : Le gradient de perte est ensuite analysé par l'algorithme de **Descente de Gradient** (*Gradient Descent*), qui choisit la direction de la pente pour ajuster les poids et ainsi réduire la perte lors de l'exécution de l'époque suivante.

Analogie :

Imaginez que vous essayez de naviguer dans le noir dans une pièce inconnue (le processus d'apprentissage) :

- Propagation Avant** est le fait de **faire un pas** (passer les données dans le modèle) et de **heurter un mur** (obtenir une prédiction).
- Fonction de Perte** est la **douleur** que vous ressentez en heurtant le mur (la mesure de l'erreur).
- La **Rétropropagation** est l'étape où vous **analysez d'où vient la douleur** (calculer le gradient de perte) et vous **déterminez quel muscle vous devez ajuster** (les poids) pour ne pas heurter le mur la prochaine fois, guidé par la Descente de Gradient.

PROCESSUS D'APPRENTISSAGE D'UN RESEAU DE NEURONES

Le processus d'apprentissage d'un réseau de neurones se déroule en **trois étapes** principales. Ce cycle est répété pour un nombre défini d'**époques** (*epochs*).
Voici les différentes phases du processus d'apprentissage :

Phase 1 : Propagation Avant (Forward Propagation)

La première étape consiste à **prendre les données d'entraînement et à appliquer la propagation avant**.

- Les données d'entraînement sont introduites dans le réseau, traversant chaque couche de l'entrée vers la sortie.
- Cette phase produit une prédiction de sortie.

Phase 2 : Rétropropagation de la Perte (Backpropagation)

C'est l'étape où le modèle évalue son erreur pour préparer les ajustements.

- L'objectif de la Rétropropagation est d'agir comme un algorithme d'apprentissage pour **améliorer les sorties** du réseau.
- Elle **corrige l'erreur** en utilisant la **fonction de perte** (*loss function*).
 - La fonction de perte mesure le niveau d'erreur à chaque époque.
- L'objectif principal est d'obtenir le **gradient de perte de chaque coefficient**.
- Pour réaliser ce calcul, la **règle de la chaîne** (*chain rule*) est appliquée : le gradient est calculé pour tous les poids et pour toutes les couches, **de la dernière à la première**.
- Le but est de calculer l'impact de chaque paramètre.

Phase 3 : Mise à Jour des Coefficients

La troisième étape utilise les résultats de la rétropropagation pour modifier la configuration du réseau afin d'améliorer la performance lors de la prochaine itération.

- Il s'agit d'**utiliser le gradient pour mettre à jour les coefficients du réseau**.
- Ceci est réalisé par l'algorithme de **Descente de Gradient** (*Gradient Descent*), qui est utilisé pour calculer l'évolution de la fonction en choisissant la direction de la pente.
- Les poids sont ajustés pour exécuter l'époque suivante.
- La perte est minimisée à travers toutes les itérations.
- Le *Batch size* définit le nombre d'échantillons à traiter avant de mettre à jour les paramètres internes du modèle.

Ce processus est répété pour le nombre d'époques défini, sachant que l'algorithme d'apprentissage parcourra l'ensemble du jeu de données d'entraînement (*training dataset*) à chaque époque.

III. Fonctions d'Activation

Notion	Définition / Explication
Fonction d'Activation	Appartient au cœur des modèles de <i>deep learning</i> . Elle applique des transformations non-linéaires aux données.
ReLU (Rectified Linear Unit)	Fonction la plus utilisée. Elle applique un filtre en gardant uniquement les valeurs positives. Équation : $f(x) = \max(0, x)$. AlexNet l'a utilisée car c'est plus rapide que la fonction tanh. (ALEXNET) La ReLU (<i>Rectified Linear Unit</i>) est la fonction d'activation la plus utilisée en <i>Deep Learning</i>. Voici son fonctionnement le plus simplement possible : Rôle : Elle s'applique à la sortie d'une couche pour corriger les données en introduisant une transformation non-linéaire. Principe : Elle applique un filtre en conservant uniquement les valeurs positives. Fonctionnement simple : Elle supprime les valeurs négatives, ou les remplace par 0. Formule : La fonction ReLU est définie par $f(x)=\max(0,x)$. Elle a été utilisée dans AlexNet et Unet pour remplacer la fonction tanh, ce qui a permis un calcul plus rapide
Leaky ReLU (Leaky Rectified Linear Unit)	Variante de ReLU qui garde les neurones actifs : $F(x) = \max(0.1x, x)$. Souvent choisie dans les tâches de segmentation.
SoftMax	Utilisée pour la classification multi-classes. Elle prend un vecteur de scores en entrée et produit un vecteur de probabilités en sortie, où la somme des valeurs est égale à 1. (Fin de la propagation avant de alexnet) Équation : $\exp(x)/\text{Somme}(\exp(x_i))$
Sigmoid	Utilisée par la dernière couche pour la classification binaire . Le résultat peut être considéré comme la probabilité d'appartenir à une classe. Équation : $1 / (1 + \exp(-x))$.
tanh (Tangente Hyperbolique)	Normalise les valeurs d'entrée entre -1 et 1. Elle peut remplacer la fonction Sigmoid pour la classification binaire sur la dernière couche. Equation : $(\exp(x)-\exp(-x))/(\exp(x)+\exp(-x))$
Softplus	Approximation douce de la fonction ReLU. Les valeurs négatives ne sont pas filtrées, mais elles tendent vers 0. Équation : $\log(\exp(x) + 1)$.

IV. Évaluation et Métriques

Notion	Définition / Explication
Loss Function (Fonction de Perte)	Fonction qui estime la perte du modèle afin que les poids puissent être mis à jour pour réduire la perte lors de l'évaluation suivante. Son but est de minimiser le taux d'erreur entre la prédiction et la vérité terrain (<i>groundtruth</i>).
Cross-Entropy / Log Loss	Fonction de perte de classification. Elle pénalise les mauvaises prédictions. Equation : $L(y,f(x))=-[y*\log(f(x))+(1-y)*\log(1-f(x))]$
Accuracy	Métrique pour mesurer la performance de l'algorithme. Calculée comme : Nombre de prédictions correctes / Nombre d'entrées .
Precision (Prec)	Mesure la proportion des Vrais Positifs (TP) parmi toutes les prédictions positives : $TP/(TP + FP)$. (FP = False Positive) -> les cas où la prédiction (<i>ypred</i>) était 1 (positif) et où la vérité terrain (<i>ytrue</i>) était 0 (négatif)
Recall (Rec)	Mesure la proportion des Vrais Positifs (TP) parmi tous les éléments positifs réels : $TP / (TP+ FN)$ (FN = False Negative) -> les cas où la prédiction (<i>ypred</i>) était 0 (négatif) et où la vérité terrain (<i>ytrue</i>) était 1 (positif)
F1 Score	Moyenne harmonique de la précision et du rappel : $(Precision*Recall)/((Precision+Recall)/2)$. Sa valeur maximale est 1 et minimale est 0.
IoU (Intersection sur Union)	Ratio de l'Intersection sur l'Union. Utilisé comme seuil pour définir si un résultat est un vrai positif ou un faux positif.
mAP (Mean Average Precision)	Calcule la précision moyenne sur les valeurs de rappel entre 0 et 1. C'est la précision moyenne sur plusieurs seuils.
Dice Metric (DICE score)	Fournit une mesure de la similarité entre deux ensembles. En segmentation, il mesure la similarité entre la segmentation prédite et la vérité terrain (<i>ground truth</i>). La valeur varie entre 0 et 1.
Contingency Table	Tableau qui présente les résultats du classifieur. Il peut être utilisé à la fois pour la classification binaire et multi-classes.

V. Architectures et Tâches de Vision

Notion	Définition / Explication
AlexNet	Architecture créée en 2010 pour le challenge ILSVRC, qu'elle a remporté en 2012 avec un taux d'erreur de 15,3%. Composée de 8 couches, elle utilise des couches de Dropout et des ReLU .
VGG Family	Architectures VGG16 et VGG19 (2014). L'idée principale est de réduire la taille du noyau de convolution à 3x3 et d'ajouter plus de couches de convolution.
ResNet (Residual Network)	Architecture introduite en 2015. Elle introduit des skip connections (<i>connexions de saut</i>) pour établir des raccourcis pour le gradient. Elle est utilisée comme <i>backbone</i> principal dans beaucoup d'architectures.
Skip Connections	Connexions qui fusionnent les caractéristiques de bas niveau et de haut niveau. Dans ResNet, elles permettent de contourner les couches pour résoudre les problèmes de gradient. Equation : $y=F(x)+x$
Backbone	Partie du réseau chargée d'extraire les caractéristiques visuelles des images. Considéré comme "l'œil du modèle".
Segmentation Sémantique	Processus qui divise une image en multiples segments ou régions. Il fournit du contexte et du sens aux pixels.
U-Net	Architecture de l'état de l'art pour la segmentation sémantique. Caractérisée par sa structure en forme de U, avec un chemin d'encodage (<i>downsampling</i>) et un chemin de décodage (<i>upsampling</i>).
Yolo (You Only Look Once)	Modèle introduit en 2015 pour la détection d'objets. Son idée principale est de ne regarder chaque pixel qu'une seule fois pour prédire la position des objets. Il utilise une grille, un <i>backbone</i> , des <i>anchor boxes</i> et le post-traitement NMS .
NMS (Non-Max Suppression)	Algorithme de post-traitement utilisé par Yolo qui permet de ne conserver que les "meilleures" boîtes englobantes.
GAN (Generative Adversarial Networks)	Réseau génératif basé sur l'apprentissage antagoniste (<i>adversarial learning</i>). Composé d'un Generator (génère des données) et d'un Discriminator (distingue les fausses données).
DCGAN (Deep Convolutional Generative Adversarial Network)	Variante de GAN qui utilise des CNN à la place des <i>Multi-Layer Perceptrons</i> (MLPs) pour produire de nouvelles images.
Synthetic Data (Données Synthétiques)	Nouvelles données produites par des modèles comme les GANs, qui ressemblent aux distributions de données réelles.

VI. Architectures NLP (Natural Language Processing)

Notion	Définition / Explication
NLP (Natural Language Processing)	Domaine lié à la compréhension du langage naturel.
NLG (Natural Language Generation)	Domaine lié à la génération de contenu en langage naturel.
RNN (Recurrent Neural Network)	Construit pour résoudre des problèmes où la séquence est plus importante que les éléments individuels. Utilise un état caché (<i>hidden state</i>) mis à jour en continu pour capturer l'information des pas de temps précédents.
LSTM (Long Short Time Memory)	Type de RNN qui introduit une mémoire cellulaire . Résout le problème de la disparition du gradient (<i>vanishing gradient</i>) des RNN. Utilise trois portes (<i>gates</i>) : <i>input</i> , <i>output</i> et <i>forget</i> .
GRU (Gate Recurrent Unit)	Amélioration du LSTM qui combine les portes <i>forget</i> et <i>input</i> en une seule.
Transformer	Architecture fortement liée au concept d' Attention . Utilise un système Encodeur/Décodeur.
Mécanisme d'Attention (<i>Attention Mechanism</i>)	Principe clé : trouver quelles parties de l'entrée sont les plus importantes et dire au modèle de s'y concentrer. Utilise trois concepts : Query (requête), Key (clé) et Value (valeur).
Self-Attention	Module où une seule séquence est utilisée et où le score de chaque mot dépend des autres mots de la même phrase.
BERT (Bidirectional Encoder Representations from Transformers)	Modèle basé sur l'encodeur du Transformer. Caractéristique : se concentre sur la compréhension du contexte d'un mot en regardant à la fois son contexte à gauche et à droite (bidirectionnel).
GPT (Generative Pre-trained Transformer)	Modèle basé sur le décodeur du Transformer. Caractéristique : utilisé pour la génération de texte . Exemple : GPT-3 est un modèle massif de 175 milliards de paramètres.
LLMs (Large Language Models)	Modèles souvent basés sur l'architecture Transformer. Utilisés pour un large éventail de tâches de NLP et NLG. Exemples : GPT-3, BERT, LLaMA.
ViT (Vision Transformer)	Modèle qui adapte l'architecture Transformer pour traiter les images, en traitant les <i>patches</i> d'images comme des données séquentielles.

Utilisation des formules :

II. Fonctions d'Activation

Les fonctions d'activation appliquent des transformations non-linéaires aux données.

2. ReLU (Rectified Linear Unit)

Formule	Méthodologie	Exemple d'utilisation simple
$f(x) = \max(0, x)$	Méthodologie : La fonction prend une valeur d'entrée (x). Si x est positif, elle garde la valeur de x ; si x est négatif, elle remplace la valeur par 0, car elle applique un filtre en gardant uniquement les valeurs positives.	Énoncé : Calculer la sortie ReLU pour les valeurs 7 et -4. Méthode : Appliquer $f(x) = \max(0, x)$. Réponse : Pour $x=7$, $f(7)=7$. Pour $x=-4$, $f(-4)=0$.

3. Sigmoid

Formule	Méthodologie	Exemple d'utilisation simple
$1 / (1 + \exp(-x))$	Méthodologie : La fonction est utilisée par la dernière couche pour la classification binaire. Elle convertit la valeur d'entrée (x) en un résultat entre 0 et 1, qui peut être considéré comme la probabilité d'appartenir à la classe positive.	Énoncé : Un neurone de sortie produit un score de 0 pour la classification binaire. Méthode : Calculer la probabilité avec $\text{Sigmoid}(0) = 1 / (1 + \exp(-0))$. Réponse : $\text{Sigmoid}(0) = 1 / (1 + 1) = 0.5$.

4. SoftMax

Formule	Méthodologie	Exemple d'utilisation simple
$\exp(x) / \sum(\exp(x_i))$	Méthodologie : Utilisée pour la classification multi-classes. Elle prend un vecteur de scores en entrée et produit un vecteur de probabilités en sortie, où chaque sortie est entre 0 et 1 et la somme totale est égale à 1.	Énoncé : Un modèle doit classer une entrée en 3 catégories. Méthode : Appliquer SoftMax aux scores bruts de la dernière couche. Réponse : La fonction produit un vecteur, par exemple [0.8, 0.15, 0.05], où la somme est 1, chaque valeur étant associée à une classe.

5. Softplus

Formule	Méthodologie	Exemple d'utilisation simple
$\log(\exp(x) + 1)$	Méthodologie : C'est une approximation douce de la fonction ReLU. Elle est calculée en appliquant la fonction logarithmique à la somme de l'exponentielle de x et de 1.	Énoncé : Calculer la sortie Softplus pour une valeur d'entrée négative, par exemple $x=-5$. Méthode : Appliquer $\log(\exp(-5) + 1)$. Réponse : La sortie sera positive, mais très proche de 0 (car $\exp(-5)$ est très petit), ce qui est la différence avec ReLU qui filtre la valeur à 0.

6. tanh (Tangente Hyperbolique)

Formule	Méthodologie	Exemple d'utilisation simple
$(\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x))$	Méthodologie : La fonction normalise les valeurs d'entrée. Elle peut remplacer la fonction Sigmoid pour la classification binaire dans la dernière couche.	Énoncé : Utiliser la fonction tanh pour normaliser les scores de sortie. Méthode : Appliquer la formule $\tanh(x)$. Réponse : La sortie sera dans la plage de -1 à 1, et ne pourra pas être considérée comme une probabilité.

III. Fonction de Perte (Loss)

7. Cross-Entropy/Log Loss (pour classification binaire)

Cette fonction évalue l'erreur du modèle, permettant aux poids d'être ajustés pour minimiser cette erreur.

Formule	Méthodologie	Exemple d'utilisation simple
$L(y, f(x)) = -[y \cdot \log(f(x)) + (1-y) \cdot \log(1-f(x))]$	Méthodologie : La formule calcule la perte basée sur la valeur binaire de l'étiquette (y , soit 0 ou 1) et la probabilité prédite ($f(x)$). Elle pénalise les mauvaises prédictions.	Énoncé : La vérité terrain est $y=1$. Le modèle prédit une faible probabilité de 0.1 pour cette classe positive ($f(x)=0.1$). Méthode : Remplacer les valeurs : $L = -[1 \cdot \log(0.1) + (1-1) \cdot \log(1-0.1)]$. Réponse : La valeur de perte (L) sera élevée, indiquant une erreur significative que le modèle devra corriger lors de la rétropropagation.

IV. Métriques d'Évaluation

Ces formules utilisent les quatre résultats possibles d'une classification (TP, FP, FN, TN) pour évaluer la performance.

8. Precision (Précision)

Formule	Méthodologie	Exemple d'utilisation simple
$Precision = \frac{TP}{TP+FP}$	Méthodologie : Mesure la proportion de Vrais Positifs (TP) parmi toutes les prédictions positives (TP + FP). On vérifie ainsi la fiabilité des prédictions positives du modèle.	Énoncé : Un modèle prédit 100 arbres au total (TP + FP), mais seulement 90 étaient correctement identifiés (TP=90). Méthode : Calculer Precision = 90 / 100. Réponse : La précision est de 90% (0.9).

9. Recall (Rappel)

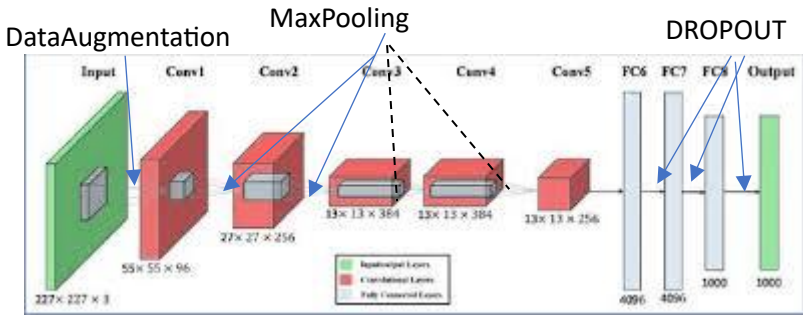
Formule	Méthodologie	Exemple d'utilisation simple
$Recall = \frac{TP}{TP+FN}$	Méthodologie : Mesure la proportion des Vrais Positifs (TP) parmi tous les éléments réellement positifs (TP + FN). On vérifie si le modèle a trouvé toutes les instances positives existantes.	Énoncé : Une image contient 100 arbres (TP + FN). Le modèle en a correctement prédit 75 (TP=75) et en a manqué 25 (FN=25). Méthode : Calculer Recall = 75 / (75 + 25). Réponse : Le Recall est égal à 75% (0.75).

10. F1 Score

Formule	Méthodologie	Exemple d'utilisation simple
$F1\ Score = \frac{Precision \times Recall}{(Precision + Recall) / 2}$	Méthodologie : C'est la moyenne harmonique de la Précision et du Rappel. Elle combine les deux métriques pour fournir une mesure équilibrée de la performance du modèle.	Énoncé : Un modèle a une Précision de 0.90 et un Recall de 0.75. Méthode : Appliquer $F1 = (0.90 \times 0.75) / ((0.90 + 0.75) / 2)$. Réponse : F1 Score = 0.825. Ce score doit être maximisé, car sa meilleure valeur est 1 et sa pire est 0.

Deux modèles appris en TD :

ALEXNET



Ceci est un résumé concis du fonctionnement et du principe d'AlexNet, adapté à une explication rapide en examen :

AlexNet : Principe et Fonctionnement

Alexnet est une architecture de réseau de neurones convolutifs (CNN) conçue pour la **classification d'images**. Le réseau est constitué de **8 couches** (incluant des couches de **Dropout**).

Fonctionnement (Propagation Avant) Le fonctionnement d'AlexNet repose sur la **Propagation Avant** (*Forward Propagation*) :

- Entrée :** Le processus commence par la préparation des données, incluant l'**Augmentation des données** (utilisée 2048 fois) pour réduire le surapprentissage.
- Extraction de Caractéristiques :** Les couches de **convolution** (avec 8 couches au total) travaillent à **extraire les caractéristiques** du volume d'entrée.
- Réduction Dimensionnelle :** Des couches de **Pooling** (spéciales avec chevauchement) sont appliquées pour **réduire les dimensions spatiales** et contrôler le surapprentissage.
- Prédiction Finale :** Le réseau se termine par une **Couche Complètement Connectée** (*Fully Connected*) dont le **but principal est d'accomplir la tâche de classification**.

Fonctionnement Détaillé de l'Architecture AlexNet (8 Couches)

L'architecture AlexNet est divisée en **huit couches principales** (cinq couches de convolution et trois couches complètement connectées). La structure est historiquement célèbre pour sa division initiale sur deux GPU et l'utilisation de techniques clés comme ReLU, Dropout, et le Pooling avec chevauchement.

I- Propagation Avant :

1. Préparation et Entrée des Données

- **Entrée** : Une image est fournie au réseau (classiquement 227x227x3).
- **Augmentation des Données** : Une technique de régularisation utilisée **2048 fois** pour créer de nouveaux échantillons et éviter le **surapprentissage (overfitting)**.

2. Étape 1 : Extraction des Caractéristiques (Conv/Pool)

Cette étape utilise les premières couches pour extraire les informations visuelles brutes .

- **Couche de Convolution (Conv)** : L'entrée est traitée par des couches de **Convolution** qui extraient les caractéristiques.
- **Activation ReLU** : Après chaque couche de convolution, la **fonction d'activation Rectified Linear Unit (ReLU)** est appliquée : $f(x) = \max(0, x)$.
 - **Avantage** : Elle permet un **calcul plus rapide** que les fonctions traditionnelles comme $\tanh$.
- **Couche de Pooling** : Une couche de **Pooling** suit pour réduire les dimensions spatiales des cartes de caractéristiques.
 - **Caractéristique Spéciale** : AlexNet utilise un **Pooling avec chevauchement** pour aider à contrôler le surapprentissage.

3. Étape 2 : Extraction et Affinement (Conv, Conv, Conv, Pool)

Les couches 2 à 5 poursuivent le processus d'extraction, capturant des caractéristiques de plus en plus complexes :

- **Séquence** : Une succession de couches de **Convolution** et de l'activation **ReLU** est utilisée.
- **Réduction** : Une couche de **Pooling** (après Conv2 et Conv5) est utilisée pour réduire les dimensions et mieux contrôler le surapprentissage.
- **Les 5 couches frontales (Conv1 à Conv5)** sont le cœur de l'extraction de caractéristiques.

4. Étape 3 : Régularisation et Classification (Les Couches FC)

Les couches finales (FC6, FC7, FC8) prennent les caractéristiques extraites et les transforment en une prédiction classifiée.

- **Couches Complètement Connectées (FC)** : Ces couches reçoivent un vecteur d'entrée (les caractéristiques finales aplaties) et ont pour but d'accomplir la tâche finale de classification.
- **Couches de Dropout** : Appliquées entre les couches FC pour la **régularisation**.
 - **Fonctionnement** : Elle **supprime aléatoirement** une partie des neurones (typiquement entre **20% et 50%**) pendant l'entraînement pour éviter que le réseau ne devienne trop dépendant de certains neurones.
- **Couche de Sortie (FC8)** : La dernière couche FC produit les scores pour chaque classe.
- 5. **Activation SoftMax** : La fonction **SoftMax** est appliquée à la sortie pour produire un **vecteur de probabilités** pour la classification multi-classes, dont la somme est égale à 1.

II- Rétropropagation

Etape 1 : Calcul de la loss

L'erreur est calculée en comparant la prédiction SoftMax à l'étiquette réelle. **Ceci est calculé après la Propagation Avant.**

Etape 2 : Le calcul du gradient

Le cœur de la Rétropropagation est le calcul des gradients (dérivées partielles) de la fonction de perte par rapport à **chaque poids** du réseau.

- **Règle de la Chaîne** : La Rétropropagation utilise la **règle de la chaîne** (un concept fondamental du calcul différentiel) pour propager l'erreur vers l'arrière, de la couche de sortie vers la couche d'entrée.
- **Sens du Flux** : Le calcul remonte des couches complètement connectées (**FC8 -> FC7 -> FC6**) vers les couches d'extraction des caractéristiques (**Conv5 -> Conv4 -> ... -> Conv1**).
- **Signification du Gradient** : Le gradient indique la direction et l'ampleur selon lesquelles chaque poids doit être modifié pour **réduire la perte**.

Etape 3 : Mise à Jour des Poids

Une fois que les gradients de tous les poids (les ≈ 60 millions de paramètres d'AlexNet) sont calculés, ils sont utilisés pour ajuster ces poids.

- **Algorithme d'Optimisation** : Un algorithme comme la **Descente de Gradient Stochastique (SGD)** ou ses variantes (Adam, RMSProp) est utilisé.
- **Formule Générale** : La formule de mise à jour s'apparente à :

$$\text{Nouveau Poids} = \text{Ancien Poids} - (\text{Taux d'Apprentissage} \times \text{Gradient})$$

- **Taux d'Apprentissage (learning rate) :** C'est un hyperparamètre crucial qui détermine la taille du pas pris dans la direction du gradient. Un taux trop élevé peut rendre l'apprentissage instable ; un taux trop faible le rend très lent.

Ce cycle complet (Propagation Avant -> Calcul de la Perte -> Rétropropagation -> Mise à Jour des Poids) est répété des milliers de fois (en utilisant des lots d'images) jusqu'à ce que le réseau converge vers une solution où la perte est minimale.

En Résumé pour l'Examen :

AlexNet est un CNN de **classification** (8 couches) qui a accéléré l'apprentissage en utilisant **ReLU** ($f(x) = \max(0, x)$). Il contrôle l'overfitting par l'**Augmentation des données** et le **Dropout**. Son processus de **Propagation Avant** utilise des couches de convolution pour l'extraction de caractéristiques et se termine par **SoftMax** pour générer les probabilités de classification. Ensuite on va calculer la Loss afin de pouvoir améliorer la performance de notre modèle avec la réactualisation du gradient (Rétropropagation)

CODE PYTHON ALEXNET

```
import numpy as np # Importe la bibliothèque NumPy pour les opérations numériques et les tableaux.
import torch # Importe la bibliothèque PyTorch, l'infrastructure principale pour le deep learning.
import torch.nn as nn # Importe le module 'nn' de PyTorch pour définir l'architecture du réseau neuronal (les couches).
import torch.optim as optim # Importe le module 'optim' pour les algorithmes d'optimisation (comme SGD) utilisés dans
l'entraînement.
from torchvision import datasets, transforms # Importe les modules pour les jeux de données (comme MNIST) et les transformations
d'images.
from torch.utils.data.sampler import SubsetRandomSampler # Importe un sampler pour diviser le jeu de données d'entraînement en
ensembles d'entraînement et de validation.
from sklearn.metrics import confusion_matrix # Importe un outil pour évaluer les performances de classification (non utilisé
dans le code actuel, mais importé).
import seaborn as sns # Importe Seaborn pour la visualisation de données (non utilisé dans le code actuel, mais importé).
import matplotlib.pyplot as plt # Importe Matplotlib pour la création de graphiques (non utilisé dans le code actuel, mais
importé).

# -----
# Data loaders for MNIST
# -----
# Définit une fonction pour créer les DataLoaders d'entraînement et de validation.
def get_train_valid_loader(data_dir, batch_size=64, valid_size=0.1, shuffle=True, random_seed=1):
    # Définit la transformation de normalisation (centrage et mise à l'échelle des données).
    normalize = transforms.Normalize((0.5,), (0.5,))

    # Définit la séquence de transformations pour les données d'entraînement.
    train_transform = transforms.Compose([
        transforms.Resize((227, 227)), # Redimensionne les images MNIST (28x28) à la taille d'entrée requise par AlexNet
(227x227).
        transforms.Grayscale(num_output_channels=3), # Convertit les images en niveaux de gris (1 canal) en 3 canaux (simule une
image couleur) pour AlexNet.
        transforms.ToTensor(), # Convertit l'image en tenseur PyTorch.
        normalize # Applique la normalisation.
    ])

    # Définit la séquence de transformations pour les données de validation (identique à l'entraînement ici).
    valid_transform = transforms.Compose([
        transforms.Resize((227, 227)),
        transforms.Grayscale(num_output_channels=3),
        transforms.ToTensor(),
        normalize
    ])

    # Charge l'ensemble d'entraînement MNIST (les données brutes pour les indices de sampler).
    train_dataset = datasets.MNIST(root=data_dir, train=True, download=False, transform=train_transform)
    # Charge l'ensemble de validation (utilisant le même jeu de données, les indices seront séparés ci-dessous).
    valid_dataset = datasets.MNIST(root=data_dir, train=True, download=False, transform=valid_transform)
```

```

num_train = len(train_dataset) # Obtient le nombre total d'échantillons d'entraînement.
indices = list(range(num_train)) # Crée une liste d'indices de 0 à num_train - 1.
split = int(np.floor(valid_size * num_train)) # Calcule le point de division pour la validation (10% ici).

# Mélange les indices si 'shuffle' est vrai.
if shuffle:
    np.random.seed(random_seed) # Fixe la graine pour la reproductibilité.
    np.random.shuffle(indices) # Mélange aléatoirement les indices.

train_idx, valid_idx = indices[split:], indices[:split] # Sépare les indices pour l'entraînement et la validation.
train_sampler = SubsetRandomSampler(train_idx) # Crée un sampler pour tirer des échantillons d'entraînement aléatoires.
valid_sampler = SubsetRandomSampler(valid_idx) # Crée un sampler pour tirer des échantillons de validation aléatoires.

# Crée le DataLoader d'entraînement en utilisant les samplers d'entraînement.
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, sampler=train_sampler)
# Crée le DataLoader de validation en utilisant les samplers de validation.
valid_loader = torch.utils.data.DataLoader(valid_dataset, batch_size=batch_size, sampler=valid_sampler)

return train_loader, valid_loader # Retourne les deux DataLoaders.

# Définit une fonction pour créer le DataLoader de test.
def get_test_loader(data_dir, batch_size=64, shuffle=False):
    normalize = transforms.Normalize((0.5,), (0.5,)) # Définit la transformation de normalisation.
    # Définit la séquence de transformations pour les données de test.
    transform = transforms.Compose([
        transforms.Resize((227, 227)), # Redimensionne à 227x227.
        transforms.Grayscale(num_output_channels=3), # Convertit en 3 canaux.
        transforms.ToTensor(), # Convertit en tenseur.
        normalize # Applique la normalisation.
    ])
    test_dataset = datasets.MNIST(root=data_dir, train=False, download=False, transform=transform) # Charge l'ensemble de test MNIST.
    test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=shuffle) # Crée le DataLoader de test (sans mélange par défaut).
    return test_loader # Retourne le DataLoader de test.

# -----
# Dataset loaders
# -----
# Charge les DataLoaders d'entraînement et de validation à partir du répertoire spécifié.
train_loader, valid_loader = get_train_valid_loader(data_dir='./MNIST/MNIST/raw', batch_size=64)
# Charge le DataLoader de test.
test_loader = get_test_loader(data_dir='./MNIST/MNIST/raw', batch_size=64)

# -----
# AlexNet model
# -----
# Définit la classe du modèle AlexNet, héritant de nn.Module.
class AlexNet(nn.Module):
    # Le constructeur de la classe, initialise les couches.
    def __init__(self, num_classes=10):
        super(AlexNet, self).__init__() # Appelle le constructeur de la classe parente (nn.Module).
        # --- Bloc Convolutif 1 (Conv1) ---
        self.layer1 = nn.Sequential(
            nn.Conv2d(3, 96, kernel_size=11, stride=4, padding=0), # Conv2D : 3 -> 96 canaux, filtre 11x11, pas de 4.
            nn.BatchNorm2d(96), # Normalisation de batch (ajout moderne).

```

```

nn.ReLU(), # Fonction d'activation ReLU.
nn.MaxPool2d(kernel_size=3, stride=2) # Max Pooling : filtre 3x3, pas de 2 (pooling avec chevauchement).

```

```
)
```

```
# --- Bloc Convolutif 2 (Conv2) ---
```

```
self.layer2 = nn.Sequential(
    nn.Conv2d(96, 256, kernel_size=5, stride=1, padding=2), # Conv2D : 96 -> 256 canaux, filtre 5x5, pas de 1, padding
```

```
    nn.BatchNorm2d(256), # Normalisation de batch.
```

```
    nn.ReLU(), # Fonction d'activation ReLU.
```

```
    nn.MaxPool2d(kernel_size=3, stride=2) # Max Pooling : filtre 3x3, pas de 2.
```

```
)
```

```
# --- Bloc Convolutif 3 (Conv3) ---
```

```
self.layer3 = nn.Sequential(
    nn.Conv2d(256, 384, kernel_size=3, stride=1, padding=1), # Conv2D : 256 -> 384 canaux, filtre 3x3, pas de 1, padding
```

```
    nn.BatchNorm2d(384), # Normalisation de batch.
```

```
    nn.ReLU() # Fonction d'activation ReLU.
```

```
) # Note : Pas de Max Pooling après Conv3 (conforme à AlexNet).
```

```
# --- Bloc Convolutif 4 (Conv4) ---
```

```
self.layer4 = nn.Sequential(
    nn.Conv2d(384, 384, kernel_size=3, stride=1, padding=1), # Conv2D : 384 -> 384 canaux, filtre 3x3, pas de 1, padding
```

```
    nn.BatchNorm2d(384), # Normalisation de batch.
```

```
    nn.ReLU() # Fonction d'activation ReLU.
```

```
) # Note : Pas de Max Pooling après Conv4 (conforme à AlexNet).
```

```
# --- Bloc Convolutif 5 (Conv5) ---
```

```
self.layer5 = nn.Sequential(
    nn.Conv2d(384, 256, kernel_size=3, stride=1, padding=1), # Conv2D : 384 -> 256 canaux, filtre 3x3, pas de 1, padding
```

```
    nn.BatchNorm2d(256), # Normalisation de batch.
```

```
    nn.ReLU(), # Fonction d'activation ReLU.
```

```
    nn.MaxPool2d(kernel_size=3, stride=2) # Max Pooling final : filtre 3x3, pas de 2.
```

```
)
```

```
# --- Couche Fully Connected 6 (FC6) ---
```

```
self.fc = nn.Sequential(
    nn.Dropout(0.5), # Ajout de Dropout (50%) pour la régularisation.
    nn.Linear(9216, 4096), # Couche linéaire (Complètement Connectée) : 9216 entrées -> 4096 sorties.
    nn.ReLU() # Fonction d'activation ReLU.
```

```
)
```

```
# --- Couche Fully Connected 7 (FC7) ---
```

```
self.fc1 = nn.Sequential(
    nn.Dropout(0.5), # Ajout de Dropout (50%) pour la régularisation.
    nn.Linear(4096, 4096), # Couche linéaire : 4096 entrées -> 4096 sorties.
    nn.ReLU() # Fonction d'activation ReLU.
```

```
)
```

```
# --- Couche Fully Connected 8 (FC8 / Sortie) ---
```

```
self.fc2 = nn.Linear(4096, num_classes) # Couche linéaire finale : 4096 entrées -> 10 sorties (classes MNIST).
```

```
# Définit le processus de propagation avant (Forward Pass).
```

```
def forward(self, x):
```

```
    out = self.layer1(x) # Passe par le Bloc Conv1.
```

```
    out = self.layer2(out) # Passe par le Bloc Conv2.
```

```
    out = self.layer3(out) # Passe par le Bloc Conv3.
```

```
    out = self.layer4(out) # Passe par le Bloc Conv4.
```

```
    out = self.layer5(out) # Passe par le Bloc Conv5.
```

```
    out = out.reshape(out.size(0), -1) # Aplattit la sortie Conv en un vecteur avant de passer aux couches FC. (Ex : de
```

```
6x6x256 à 9216)
```

```

out = self.fc(out) # Passe par la Couche FC6.
out = self.fc1(out) # Passe par la Couche FC7.
out = self.fc2(out) # Passe par la Couche FC8 (sortie finale, les logits).
return out # Retourne la sortie finale (les scores de classe avant SoftMax).

# -----
# Hyperparameters
# -----
num_classes = 10 # Nombre de classes de sortie (0-9 pour MNIST).
num_epochs = 10 # Nombre de fois où l'ensemble des données d'entraînement sera parcouru.
learning_rate = 0.005 # Taux d'apprentissage pour l'optimiseur.

model = AlexNet(num_classes).to(device) # Initialise le modèle AlexNet et le déplace vers le périphérique (GPU/CPU).
criterion = nn.CrossEntropyLoss() # Définit la fonction de perte (Loss Function) pour la classification multi-classes.
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0.9, weight_decay=0.005) # Définit l'optimiseur
(Descente de Gradient Stochastique avec momentum et régularisation L2).

# -----
# Training loop
# -----
total_step = len(train_loader) # Calcule le nombre total d'itérations (batchs) par epoch.

# Boucle principale d'entraînement.
for epoch in range(num_epochs):
    model.train() # Met le modèle en mode entraînement (active Dropout et BatchNorm).
    running_loss = 0.0 # Initialise la perte cumulée pour l'epoch.
    # Boucle pour itérer sur les batchs de données d'entraînement.
    for i, (images, labels) in enumerate(train_loader):
        images = images.to(device) # Déplace les images vers le périphérique.
        labels = labels.to(device) # Déplace les étiquettes vers le périphérique.

        # Forward
        outputs = model(images) # Effectue la propagation avant pour obtenir les prédictions.
        loss = criterion(outputs, labels) # Calcule la perte entre les prédictions et les étiquettes réelles.

        # Backward
        optimizer.zero_grad() # Remet les gradients des paramètres à zéro (étape nécessaire avant la rétropropagation).
        loss.backward() # Effectue la rétropropagation (calcul des gradients de la perte par rapport aux poids).
        optimizer.step() # Met à jour les poids du modèle en utilisant l'optimiseur.

    running_loss += loss.item() # Ajoute la perte du batch actuel à la perte cumulée.

print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {running_loss/total_step:.4f}") # Affiche la perte moyenne pour l'epoch.

# -----
# Validation
# -----
model.eval() # Met le modèle en mode évaluation (désactive Dropout et utilise les moyennes pour BatchNorm).
# Désactive le calcul du gradient (optimisation de la mémoire et de la vitesse pour l'évaluation).
with torch.no_grad():
    correct = 0 # Compteur des prédictions correctes.
    total = 0 # Compteur total des échantillons.
    # Boucle pour itérer sur les batchs de données de validation.
    for images, labels in valid_loader:
        images = images.to(device)
        labels = labels.to(device)

```

```
outputs = model(images) # Propagation avant.  
_, predicted = torch.max(outputs.data, 1) # Obtient la classe prédite (indice de la valeur maximale).  
total += labels.size(0) # Ajoute la taille du batch au total.  
correct += (predicted == labels).sum().item() # Compte le nombre de prédictions correctes.
```

```
print(f'Validation Accuracy: {100 * correct / total:.2f}%') # Affiche la précision (Accuracy) de validation pour  
l'epoch.
```

```
# -----
```

```
# Test set evaluation
```

```
# -----
```

```
model.eval() # Met le modèle en mode évaluation.
```

```
# Désactive le calcul du gradient.
```

```
with torch.no_grad():
```

```
    correct = 0
```

```
    total = 0
```

```
    # Boucle pour itérer sur les batchs de données de test.
```

```
    for images, labels in test_loader:
```

```
        images = images.to(device)
```

```
        labels = labels.to(device)
```

```
        outputs = model(images)
```

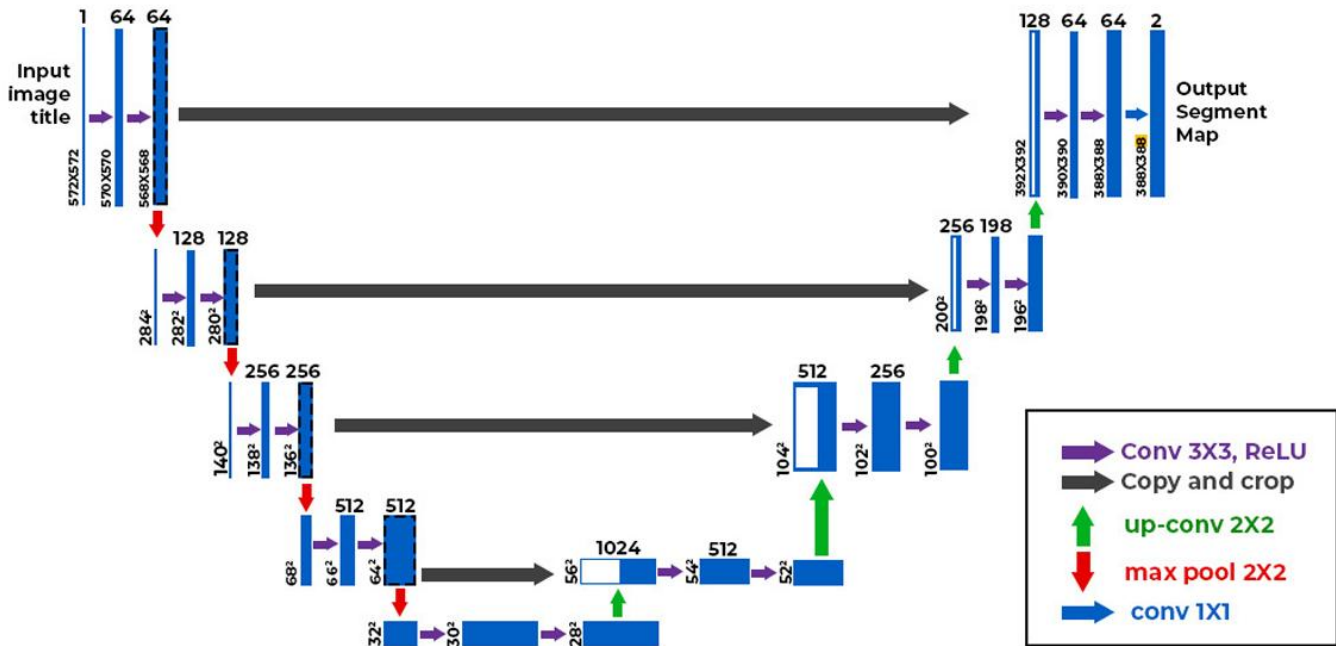
```
        _, predicted = torch.max(outputs.data, 1)
```

```
        total += labels.size(0)
```

```
        correct += (predicted == labels).sum().item()
```

```
print(f'Test Accuracy: {100 * correct / total:.2f}%') # Affiche la précision finale sur l'ensemble de test.
```

UNET



L'Architecture U-Net : Descriptif Détaillé

L'architecture **U-Net** est un **réseau de neurones convolutifs (CNN)** considéré comme l'état de l'art dans son domaine, conçu spécifiquement pour la **segmentation sémantique d'images**.

I. Objectif et Rôle Principal

- **Tâche** : La **segmentation sémantique** consiste à diviser une image en multiples segments pour attribuer du contexte et du sens à chaque pixel individuel.
- **Résultat** : Le modèle doit produire une **carte de segmentation (segmentation map)** qui a les **mêmes dimensions spatiales** que l'image d'entrée.
- **Applications** : Principalement utilisée en **analyse d'images médicales** (segmentation de tumeurs) et en **détection d'objets** (cartographie satellite).

II. Architecture en "U" : Encoding et Decoding Paths

L'architecture U-Net est caractérisée par sa structure symétrique en forme de 'U', divisée en deux chemins principaux :

1. 📌 Chemin d'Encodage (Encoding Path / Chemin de Contraction)

Ce chemin agit comme un extracteur de caractéristiques (similaire à un classifieur) en réduisant progressivement la taille des données spatiales pour **capturer le contexte**.

Étape Clé	Opérations du Bloc	Rôle et Mécanisme
Extraction	Double Convolution : Deux conv 3x3 successives, chacune suivie de ReLU .	Extrait des caractéristiques riches et complexes au sein d'un même bloc.
Downsampling	Max-pooling 2x2 .	Réduit les dimensions spatiales par un facteur de deux. Cela augmente le champ de vision des filtres pour capturer un contexte plus large .

Étape Clé	Opérations du Bloc	Rôle et Mécanisme
Sauvegarde	Connexion de Saut (Skip Connection).	Le tenseur convolué est sauvegardé juste avant le Max-pooling pour être utilisé plus tard dans le chemin de décodage.

2. Chemin de Décodage (Decoding Path / Chemin d'Expansion)

Ce chemin est chargé de reconstruire la carte de segmentation en "étendant" les caractéristiques contractées pour assurer une **localisation précise** des objets.

Étape Clé	Opérations du Bloc	Rôle et Mécanisme
Upsampling	Déconvolution conv 2x2 via ConvTranspose2d.	Augmente la taille spatiale pour reconstruire la résolution de l'image.
Fusion (Innovation Clé)	Connexion de Saut : copy and crop (copier et rogner) et concaténation (torch.cat).	Les caractéristiques sauvegardées de l'encodeur sont fusionnées avec celles de l'Upsampling. Cette étape incorpore des détails spatiaux de haute résolution qui ont été perdus.
Affinement	Double Convolution : Deux couches conv 3x3 consécutives, suivies de ReLU.	Affine les caractéristiques combinées (contexte + localisation) pour générer une segmentation précise.

III. Les Connexions de Saut (L'Innovation Clé)

Les *Skip Connections* sont l'élément fondamental de l'U-Net qui garantit la haute précision, ce qui rend cette architecture supérieure pour la segmentation.

- Localisation Précise** : On entend par là la capacité du réseau à identifier et à **délimiter les contours exacts** d'un objet **au niveau du pixel**, ce qui s'oppose à la détection (boîte englobante) ou à la classification (étiquette unique).
- Rôle Crucial** : Dans les CNN classiques, les étapes de *Max-pooling* (Downsampling) réduisent la taille spatiale pour capturer le contexte, ce qui rend les détails fins des contours flous (perte de localisation).
- Mécanisme** : Les *skip connections* transfèrent directement les **caractéristiques de haute résolution** (information locale) du Chemin d'Encodage vers le Chemin de Décodage, permettant à ce dernier de recevoir l'information **contextuelle** (du bas du 'U') fusionnée avec l'information **locale et précise**.

IV. Processus d'Apprentissage et Sortie

Le processus d'apprentissage d'U-Net suit le cycle standard des réseaux de neurones, mais utilise des métriques spécifiques :

Étape	Processus	Rôle et Mécanisme
Couche de Sortie	Convolution $\times 1$ $\times 1$	Mappe le vecteur de caractéristiques final au nombre de classes de segmentation (ex. 2 pour fond et objet), produisant la carte de segmentation finale.
Propagation Avant	Flux de données	Les données traversent les chemins d'encodage et de décodage pour obtenir la carte de segmentation prédite.
Rétropropagation de la Perte	Apprentissage	La perte est calculée et rétropropagée (<code>loss.backward()</code>) pour obtenir le gradient de chaque coefficient.
Évaluation	Métrique DICE	Utilisée pour mesurer la similarité entre la segmentation prédite et la vérité terrain (<i>ground truth</i>). Un score DICE de 1 signifie un chevauchement parfait.

Comparaison : Convolution vs. Double Convolution

Caractéristique	Convolution (Simple)	Double Convolution
Opération	Une seule application d'un filtre convolutif.	Deux applications consécutives de filtres convolutifs.
Séquence Type	Conv ->ReLU ->Pool	Conv ->ReLU ->Conv ->ReLU ->Pool
Objectif	Extraire les caractéristiques à un niveau donné.	Extraire des caractéristiques plus complexes et profondes dans un même bloc sans perdre de résolution spatiale.
Exemple d'Usage	Architectures classiques (ex: AlexNet).	Caractéristique des architectures de segmentation (U-Net).

Programme UNET :

```
class DoubleConv(nn.Module): # Définition d'une classe qui hérite de nn.Module (base pour tous les modules PyTorch)
    def __init__(self, in_channels, out_channels): # Constructeur : définit le nombre de canaux d'entrée et de sortie
        super().__init__() # Appel du constructeur de la classe parente nn.Module pour initialiser correctement le module
        # Définition de la séquence d'opérations pour la double convolution
        self.conv_op = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1), # 1ère convolution 2D :
# in_channels = nombre de canaux d'entrée (ex: 3 pour RGB)
# out_channels = nombre de canaux de sortie
# kernel_size=3 => filtre 3x3
# padding=1 => conserve la taille spatiale
            nn.ReLU(inplace=True), # Activation ReLU après la première convolution (remplace les valeurs négatives par 0)
                # inplace=True -> modifie le tenseur en place pour économiser la mémoire
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1), # 2ème convolution 3x3 sur les canaux de sortie de la 1ère
convolution
            nn.ReLU(inplace=True) # ReLU après la 2ème convolution
        )
    def forward(self, x): # Méthode forward : définit comment les données passent dans le module suivant
        return self.conv_op(x) # Applique la séquence conv+ReLU au tenseur d'entrée x et renvoie le résultat

# ----- Downsample / Encoding Path -----
# Le downsampling correspond à la partie gauche de l'U-Net (chemin d'encodage).
# Ici, on fait deux choses principales pour chaque niveau :
# 1) Double Convolutions : on applique deux convolutions consécutives pour extraire des caractéristiques importantes de l'image.
# 2) Max Pooling : on réduit la taille de l'image (largeur et hauteur) pour que le réseau apprenne des motifs plus globaux et pour diminuer le coût
de calcul.
# Les informations extraites avant le pooling sont conservées pour les "skip connections" qui seront utilisées plus tard dans le décodage.
class DownSample(nn.Module): # Définition d'une classe PyTorch pour une étape de downsampling
    def __init__(self, in_channels, out_channels): # Constructeur avec le nombre de canaux en entrée et sortie
        super().__init__() # Initialise la classe parente nn.Module
        self.conv = DoubleConv(in_channels, out_channels)
        # Crée un bloc de Double Convolutions pour extraire des caractéristiques de l'image
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        # Crée un MaxPooling 2x2 pour réduire la taille spatiale de l'image (downsampling)
    def forward(self, x): # Définition de la propagation avant
        down = self.conv(x)
        # Applique les convolutions et ReLU : on obtient un tenseur enrichi en features
        # Tensor : c'est juste la façon dont l'IA stocke et traite les données numériques.
        # Features : caractéristiques visuelles extraites par les convolutions à partir des tenseurs d'entrée.
        p = self.pool(down)
        # Applique le MaxPooling sur ce tenseur pour réduire ses dimensions (réduit largeur et hauteur de moitié)
        return down, p
        # Retourne deux tenseurs :
        # down -> le tenseur convolué avant pooling (sera utilisé pour les skip connections dans le décodage)
        # p -> le tenseur réduit après pooling (sera l'entrée pour le niveau suivant dans l'encodage)

# ----- Upsample / Decoding Path -----
# La partie upsampling (décodage) correspond à la droite de la figure :
# - On commence par une dé-convolution (ou upsampling) pour augmenter la taille du tenseur et ainsi reconstruire les zones segmentées.
# - Ensuite on applique une double convolution pour raffiner les features.
# - Avant chaque MaxPooling dans l'encodage, on copie et recadre le tenseur pour les skip connections
# afin de le concaténer avec le tenseur upsample.
class UpSample(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        # Définition de la couche de dé-convolution (ConvTranspose2d)
        # Elle va doubler la taille spatiale du tenseur (upsampling)
```

```

# in_channels : nombre de canaux en entrée
# in_channels//2 : nombre de canaux en sortie
# kernel_size=2, stride=2 : paramètres pour doubler la dimension spatiale
self.up = nn.ConvTranspose2d(in_channels, in_channels//2, kernel_size=2, stride=2)

# Définition d'une double convolution pour traiter le tenseur concaténé
# in_channels : après concaténation avec le tenseur skip, le nombre de canaux est doublé
# out_channels : nombre de canaux souhaité en sortie de cette étape
self.conv = DoubleConv(in_channels, out_channels)

def forward(self, x1, x2):
    # x1 : tenseur venant de la couche précédente (plus petit, upsample ici)
    # x2 : tenseur provenant de la skip connection correspondante dans l'encodage
    x1 = self.up(x1) # Applique l'upsampling sur x1 pour retrouver la dimension spatiale de x2
    x = torch.cat([x1, x2], 1) # Concatène les deux tenseurs sur la dimension des canaux (dim=1)
    return self.conv(x) # Applique la double convolution sur le tenseur concaténé pour affiner les features

# ----- ARCHITECTURE UNET -----

class UNet(nn.Module):
    def __init__(self, in_channels, num_classes): #On crée les couches
        super().__init__()
        # ----- ENCODING (Downsampling) -----
        # Chaque DownSample fait DoubleConv + MaxPooling
        # Le tenseur "down" sera sauvegardé pour la skip connection
        self.down_convolution_1 = DownSample(in_channels, 64) # 1er bloc : entrée -> 64 canaux
        self.down_convolution_2 = DownSample(64, 128) # 2e bloc : 64 -> 128 canaux
        self.down_convolution_3 = DownSample(128, 256) # 3e bloc : 128 -> 256 canaux
        self.down_convolution_4 = DownSample(256, 512) # 4e bloc : 256 -> 512 canaux
        # Chaque encodage a capturé des infos différentes et apprend plus de détails et va aider pour la reconstruction :
        # Les premiers encodages -> petits détails
        # Les derniers encodages -> formes générales

        # ----- BOTTLE NECK -----
        # Après les 4 downsamplings, l'image est très petite en taille mais avec beaucoup de canaux
        # -> elle contient les informations les plus abstraites et globales.
        # Le bottleneck fait juste une double convolution pour traiter ces infos profondes sans réduire davantage la taille.
        # Il prépare ensuite le décodage (upsampling) pour reconstruire l'image avec toutes les informations capturées.
        # Partie la plus profonde de l'UNet, DoubleConv uniquement (Partie basse du V)
        # Pas de MaxPooling ici (on ne réduit pas la taille d'avantage)
        self.bottle_neck = DoubleConv(512, 1024) # 512 -> 1024 canaux

        # ----- DECODING (Upsampling) -----
        # Chaque UpSample fait ConvTranspose2d (upsample) + concat skip connection + DoubleConv
        # Chaque upsampling reconstruit l'image progressivement.
        # Les skip connections permettent de garder les détails fins perdus lors des downsamplings et les transférer aux upsampling.
        # La double convolution après chaque upsample sert à affiner et mélanger les infos anciennes (skip) et nouvelles (upsampled)
        self.up_convolution_1 = UpSample(1024, 512) # 1024 -> 512 canaux, fusion avec down_4
        self.up_convolution_2 = UpSample(512, 256) # 512 -> 256 canaux, fusion avec down_3
        self.up_convolution_3 = UpSample(256, 128) # 256 -> 128 canaux, fusion avec down_2
        self.up_convolution_4 = UpSample(128, 64) # 128 -> 64 canaux, fusion avec down_1

        # ----- OUTPUT -----
        # Convolution 1x1 pour réduire les 64 canaux en "num_classes" canaux de sortie afin de ne garder que ce qu'on "prédit"
        self.out = nn.Conv2d(in_channels=64, out_channels=num_classes, kernel_size=1)

    def forward(self, x): #On utilise les couches générées dans le __init__

```

```

# ----- ENCODING -----
down_1, p1 = self.down_convolution_1(x) # 1er bloc : convolutions + maxpool
down_2, p2 = self.down_convolution_2(p1) # 2e bloc
down_3, p3 = self.down_convolution_3(p2) # 3e bloc
down_4, p4 = self.down_convolution_4(p3) # 4e bloc

# ----- BOTTLE NECK -----
b = self.bottle_neck(p4) # convolutions profondes sans pooling

# ----- DECODING -----
# Chaque bloc d'upsample prend le tenseur upsample + tenseur de skip connection
up_1 = self.up_convolution_1(b, down_4) # fusion avec down_4
up_2 = self.up_convolution_2(up_1, down_3) # fusion avec down_3
up_3 = self.up_convolution_3(up_2, down_2) # fusion avec down_2
up_4 = self.up_convolution_4(up_3, down_1) # fusion avec down_1

# ----- OUTPUT -----
# Convolution finale pour produire le nombre de classes souhaité
out = self.out(up_4)
return out

# ----- MODÈLE -----
model = UNet(in_channels=3, num_classes=1).to(device)
# Crée un modèle UNet pour segmentation binaire
# in_channels=3 car les images sont RGB
# num_classes=1 car on prédit un masque binaire
# .to(device) : envoie le modèle sur GPU si disponible

# ----- OPTIMIZER -----
optimizer = optim.AdamW(model.parameters(), lr=3e-4)
# Optimizer AdamW : met à jour les poids du modèle en utilisant le learning rate défini

# ----- LOSS -----
criterion = nn.BCEWithLogitsLoss()
# Loss pour segmentation binaire
# Combine Sigmoid + Binary Cross Entropy, adaptée aux sorties binaires

# ----- DICE METRIC -----
# Permet de mesurer à quel point le masque prédit correspond au masque réel

model = UNet(in_channels=3, num_classes=1).to(device)
optimizer = optim.AdamW(model.parameters(), lr=3e-4)
criterion = nn.BCEWithLogitsLoss()

def dice_coefficient(prediction, target, epsilon=1e-07):
    """
    Calcul du Dice coefficient pour mesurer la qualité de segmentation.
    y_true : masque réel
    y_pred : masque prédit par le modèle
    smooth : constante pour éviter la division par zéro
    """
    prediction_copy = prediction.clone()

    # Binarisation : tous les pixels <0 deviennent 0
    prediction_copy[prediction_copy < 0] = 0
    # Binarisation : tous les pixels >0 deviennent 1
    prediction_copy[prediction_copy > 0] = 1

```

```

# Calcule le nombre de pixels corrects (intersection)
intersection = abs(torch.sum(prediction_copy * target))

# Calcule la somme totale des pixels blancs dans les deux masques (union)
union = abs(torch.sum(prediction_copy) + torch.sum(target))

# Formule du Dice coefficient avec epsilon pour éviter la division par zéro
dice = (2. * intersection + epsilon) / (union + epsilon)

# Retourne le Dice coefficient
return dice

generator = torch.Generator().manual_seed(42)
dataset_full = PetSegmentationDataset(DATA_DIR)
train_size = int(0.8 * len(dataset_full))
val_size = len(dataset_full) - train_size
train_dataset, val_dataset = random_split(dataset_full, [train_size, val_size], generator=generator)
EPOCHS = 10 # Nombre de fois que l'on va passer sur tout le dataset (10 itérations complètes)

# Pour le Dice coefficient de validation
BATCH_SIZE = 8
num_workers = 2

train_dataloader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=num_workers)
val_dataloader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=num_workers)

if __name__ == "__main__":

    # Listes pour stocker l'historique des pertes (loss) et du Dice coefficient
    train_losses = [] # Pour la perte d'entraînement
    train_dices = [] # Pour le Dice coefficient d'entraînement
    val_losses = [] # Pour la perte de validation
    val_dices = []

    # ----- DATALOADERS -----

    # DataLoader pour l'entraînement
    train_dataloader = DataLoader(
        dataset=train_dataset, # Dataset d'entraînement
        num_workers=num_workers, # Nombre de processus parallèles pour charger les données
        pin_memory=False, # Si True, les données sont préchargées dans la mémoire GPU pour accélérer
        batch_size=BATCH_SIZE, # Taille des batches
        shuffle=True # Mélange aléatoire des données à chaque epoch pour éviter l'ordre fixe
    )

    # DataLoader pour la validation
    val_dataloader = DataLoader(
        dataset=val_dataset,
        num_workers=num_workers,
        pin_memory=False,
        batch_size=BATCH_SIZE,
        shuffle=True # Ici on peut aussi mettre False, mais True c'est pas gênant
    )

    # Boucle principale sur chaque epoch
    for epoch in tqdm(range(EPOCHS)):
        model.train() # Met le modèle en mode entraînement (active Dropout, BatchNorm, etc.)
        train_running_loss = 0 # Somme des pertes pour cet epoch
        train_running_dc = 0 # Somme des Dice coefficients pour cet epoch

        # Boucle sur tous les batches du DataLoader d'entraînement
        for idx, img_mask in enumerate(tqdm(train_dataloader, position=0, leave=True)):

```

```

img = img_mask[0].float().to(device) # Image du batch, convertie en float et envoyée sur GPU/CPU
mask = img_mask[1].float().to(device) # Masque réel correspondant

y_pred = model(img) # Passage avant : le modèle prédit le masque
optimizer.zero_grad() # Réinitialise les gradients avant le backward

dc = dice_coefficient(y_pred, mask) # Calcul du Dice coefficient pour ce batch
loss = criterion(y_pred, mask) # Calcul de la perte (BCEWithLogitsLoss)

train_running_loss += loss.item() # Ajout de la perte de ce batch à la somme
train_running_dc += dc.item() # Ajout du Dice de ce batch à la somme
loss.backward() # Rétropropagation pour calculer les gradients
optimizer.step() # Mise à jour des poids du modèle

# Moyenne des metrics sur tous les batches d'entraînement
train_loss = train_running_loss / (idx + 1)
train_dc = train_running_dc / (idx + 1)

# Sauvegarde des metrics d'entraînement
train_losses.append(train_loss)
train_dices.append(train_dc)

# ----- VALIDATION -----
model.eval() # Mode évaluation (désactive Dropout, BatchNorm etc.)
val_running_loss = 0
val_running_dc = 0

with torch.no_grad(): # Pas besoin de calculer les gradients pour la validation
    for idx, img_mask in enumerate(tqdm(val_dataloader, position=0, leave=True)):
        img = img_mask[0].float().to(device)
        mask = img_mask[1].float().to(device)

        y_pred = model(img) # Prédiction du masque
        loss = criterion(y_pred, mask) # Calcul de la perte
        dc = dice_coefficient(y_pred, mask) # Calcul du Dice coefficient

        val_running_loss += loss.item() # Somme des pertes sur la validation
        val_running_dc += dc.item() # Somme des Dice sur la validation

# Moyenne des metrics sur tous les batches de validation
val_loss = val_running_loss / (idx + 1)
val_dc = val_running_dc / (idx + 1)

# Sauvegarde des metrics de validation
val_losses.append(val_loss)
val_dices.append(val_dc)

# Affichage des résultats pour cet epoch
print("-" * 30)
print(f"Training Loss EPOCH {epoch + 1}: {train_loss:.4f}")
print(f"Training DICE EPOCH {epoch + 1}: {train_dc:.4f}")
print("\n")
print(f"Validation Loss EPOCH {epoch + 1}: {val_loss:.4f}")
print(f"Validation DICE EPOCH {epoch + 1}: {val_dc:.4f}")
print("-" * 30)

```

Types de questions à l'examen

- **Question 2** : Quelle devra être la taille de la dernière couche « full connected » d'un réseau de type AlexNet dont la tâche est de réaliser la classification d'images de taille 224X224 en 10 catégories sachant que l'avant-dernière couche produit un tensor de 2048.

Méthodologie

Le rôle principal des couches "**Fully Connected**" (**complètement connectées**) est d'accomplir la tâche finale du réseau, comme la classification ou l'obtention de scores.

Dans un contexte de classification, la dernière couche est responsable de la production des résultats finaux, généralement sous la forme de scores ou de probabilités pour chaque classe.

Pour une tâche de **classification multi-classes** (ici, 10 catégories), la couche de sortie doit produire un vecteur dont la dimension correspond au nombre total de classes. Cette dernière couche utilise souvent la fonction d'activation **SoftMax** qui produit un vecteur de probabilités où la somme des valeurs est égale à 1, chaque valeur étant associée à une classe.

Le fait que l'avant-dernière couche produise un tenseur de 2048 indique la taille du vecteur d'entrée pour la dernière couche, mais n'influence pas la taille de sa sortie nécessaire à la classification.

Réponse

La tâche étant la classification en **10 catégories**, la dernière couche « fully connected » devra avoir une taille de **10**. Elle produira **un vecteur de 10 valeurs**, chaque valeur représentant le score ou la probabilité que l'image appartienne à l'une des 10 catégories.

En conclusion, la "méthode de calcul" repose sur le principe suivant :

La dimension de la couche de sortie d'un réseau de classification est un paramètre défini par le nombre de classes de la tâche à accomplir.

Si le réseau doit distinguer 10 catégories, la dernière couche (la couche SoftMax) doit produire 10 probabilités, une pour chaque catégorie, d'où la taille de 10.

I. Fondamentaux et Processus d'Apprentissage

Question : Décrivez les trois étapes fondamentales du cycle d'apprentissage d'un réseau de neurones.

Réponse : 1) **Forward pass** : les données passent à travers le réseau pour produire une prédiction. 2) **Calcul de la perte** : on mesure l'écart entre la prédiction et la vérité terrain à l'aide d'une fonction de coût. 3) **Backward pass (rétropropagation)** : on calcule les gradients de la perte par rapport aux poids et on met à jour les paramètres pour améliorer la précision.

Question : Quel est le rôle de la rétropropagation ?

Réponse : La rétropropagation calcule les gradients de la fonction de perte par rapport à chaque poids du réseau. Ces gradients indiquent comment ajuster chaque poids pour réduire l'erreur, permettant au réseau de **s'auto-corriger** et de mieux apprendre.

II. Paramètres et Hyperparamètres

Question : Définissez "Epoch number" et "Batch size".

Réponse : Un **epoch** correspond à un passage complet sur l'ensemble du dataset. Le **batch size** est le nombre d'exemples traités avant chaque mise à jour des poids. Un batch trop petit peut rendre l'apprentissage instable ; un batch trop grand ralentit le calcul.

Question : Quel est le rôle du Learning Rate ?

Réponse : Le **learning rate** contrôle la taille des pas lors de la descente de gradient. Trop petit = apprentissage lent, risque de rester bloqué dans un minimum local. Trop grand = poids qui oscillent et réseau qui diverge.

III. Fonctions d'Activation

Question : Qu'est-ce que ReLU ? Donnez sa formule.

Réponse : ReLU = **Rectified Linear Unit**, formule : $f(x) = \max(0, x)$. Elle active uniquement les valeurs positives et rend le calcul plus rapide que Sigmoid/Tanh, car elle évite les exponentielles et réduit le problème de gradients nuls.

Question : Quand utiliser SoftMax plutôt que Sigmoid ?

Réponse : **SoftMax** est utilisé pour les tâches de **classification multi-classes** car il produit une probabilité pour chaque classe qui somme à 1. **Sigmoid** est adapté à la classification binaire ou multi-label où chaque classe est indépendante.

IV. Architectures CNN (AlexNet)

Question : Quel est l'objectif d'AlexNet et combien de couches a-t-il ?

Réponse : AlexNet vise à **classer des images à grande échelle** (ImageNet). Il comprend 8 couches principales : 5 convolutions + 3 fully connected.

Question : Quels sont les types de couches d'un CNN et rôle du pooling ?

Réponse : 1) **Convolution** : extrait les caractéristiques locales. 2) **Activation** (ReLU) : introduit de la non-linéarité. 3) **Pooling** : réduit la taille spatiale, conserve les informations essentielles et réduit le coût de calcul. 4) **Fully-connected** : combine toutes les features pour la décision finale.

Question : Donnez deux techniques anti-overfitting dans AlexNet.

Réponse : **Dropout** (désactive aléatoirement des neurones) et **Data Augmentation** (variation artificielle des images pour enrichir le dataset).

Question : Quel est le principe du Dropout ?

Réponse : Pendant l'entraînement, des neurones sont désactivés aléatoirement, empêchant le réseau de trop dépendre de certaines connexions et favorisant la généralisation.

V. Segmentation Sémantique (U-Net)

Question : Quel est l'objectif de U-Net et son domaine principal ?

Réponse : U-Net prédit un **masque pixel par pixel** pour segmenter des objets dans une image. Principal domaine : **imagerie médicale** (ex : segmentation d'organes ou lésions).

Question : Pourquoi U-Net a une structure en U ?

Réponse : La structure en **U** combine un **encodeur** (qui réduit la taille et extrait des caractéristiques globales) et un **décodeur** (qui reconstruit la résolution originale), formant un U.

Question : Quel est le rôle des Skip Connections ?

Réponse : Les skip connections relient les couches de l'encodeur aux couches correspondantes du décodeur pour transmettre les **détails fins perdus lors du downsampling**, ce qui améliore la précision du masque final.

Question : Que contient un module de Double Convolution ?

Réponse : Deux convolutions 3x3 consécutives avec **ReLU**, souvent suivies de **BatchNorm**, pour extraire et combiner efficacement les caractéristiques.

VI. Évaluation en Segmentation

Question : Qu'est-ce que la métrique DICE ?

Réponse : Le **Dice coefficient** mesure le chevauchement entre le masque prédit et le masque réel :

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

Score = 1 → masque parfait ; score = 0 → pas de chevauchement.

VII. Évaluation en Classification / Détection

Question : Définissez Precision et Recall.

Réponse : **Precision** = proportion de prédictions positives correctes (exactitude). **Recall** = proportion de vrais positifs détectés (couverture).

Question : Qu'est-ce que le F1-Score ?

Réponse : Moyenne harmonique de Precision et Recall. Mesure l'équilibre entre précision et recall.

♦ I. Fondamentaux CNN et Apprentissage

Question 1 : Vous avez une image 224x224 et appliquez une convolution 3x3 avec padding=1 et stride=1. Quelle sera la taille de sortie ?

Réponse : La taille reste 224x224. Avec padding=1, le bord est conservé et stride=1 ne change pas la dimension.

Réponse :

Formule : $O = W - K + 2PS + 1$

$O = 224 - 3 + 2 \times 1 + 1 = 224$

Donc la sortie est **224x224**.

Question 1.2 : Même image mais avec MaxPool 2x2, stride=2. Quelle est la taille après pooling ?

Réponse :

$O = \frac{224 - 2}{2} + 1 = 112$

Donc **112x112**. Le pooling réduit les dimensions par 2.

Question 1.3 : Vous avez un tensor 256x256 et appliquez deux blocs de pooling 2x2. Quelle sera la taille finale ?

Réponse :

1er bloc : $256 \rightarrow 128$

2e bloc : $128 \rightarrow 64$

Donc **64x64**.

Question 1.4 : Dans AlexNet, quelle est la taille de la dernière couche fully connected si l'avant-dernière couche produit un tensor de 4096 et 10 classes ?

Réponse : La dernière couche doit avoir **10 neurones**, un pour chaque classe.

Question 2 : Quel est le rôle d'une couche de **MaxPooling** ?

Réponse : Réduit la dimension spatiale, conserve les caractéristiques les plus importantes et diminue le coût de calcul. Ex : MaxPool 2x2 divise largeur/hauteur par 2.

Question 3 : Qu'est-ce que le **stride** dans une convolution ?

Réponse : Nombre de pixels par déplacement du filtre. Stride>1 réduit la taille de sortie et capture moins de détails fins.

Question 4 : Quel mécanisme permet au réseau de propager l'information d'une époque à l'autre ?

Réponse : La **rétropropagation (backpropagation)** met à jour les poids en fonction du gradient de la perte.

Question 5 : Que fait la fonction d'activation **ReLU** ?

Réponse : $f(x)=\max(0,x)$. Elle introduit la non-linéarité, permet un calcul rapide et évite le problème de gradients nuls des sigmoïdes.

♦ II. AlexNet – Architecture et Métriques

Question 6 : Quel est l'objectif principal d'AlexNet ?

Réponse : Classifier des images en plusieurs catégories. Première architecture profonde efficace sur ImageNet.

Question 7 : Combien de couches principales contient AlexNet ?

Réponse : 8 couches apprenantes : 5 convolutions + 3 fully connected.

Question 8 : Quelle est la taille de la dernière couche fully connected si l'avant-dernière produit un vecteur de 4096 et qu'on a 10 classes ?

Réponse : La dernière couche doit être de taille 10.

Question 9 : Quels types de couches composent généralement un CNN ?

Réponse : Convolution, Pooling, Activation (ReLU), Fully Connected. Pooling réduit la dimension tout en conservant les caractéristiques.

Question 10 : Quel est le rôle du **Dropout** dans AlexNet ?

Réponse : Supprime aléatoirement des neurones pendant l'entraînement pour réduire le surapprentissage.

Question 11 : Pourquoi utiliser le **BatchNorm** après une convolution ?

Réponse : Normalise les activations, accélère l'apprentissage et stabilise le réseau.

Question 12 : Quelle fonction de perte est utilisée pour la classification multi-classes ?

Réponse : **CrossEntropyLoss**, combine softmax + log loss.

♦ III. U-Net – Segmentation

Question 13 : Quel est l'objectif d'U-Net ?

Réponse : Segmenter des images pour identifier des objets/pixels spécifiques (ex : médical, satellite).

Question 14 : Pourquoi la structure est-elle en **U** ?

Réponse : Encodeur réduit la taille et extrait les features, décodeur reconstruit l'image, les skip connections relient encodeur et décodeur pour conserver les détails.

Question 15 : Que fait un module de **Double Convolution** ?

Réponse : Deux convolutions 3x3 consécutives avec ReLU, extrayant plus de caractéristiques fines à chaque niveau.

Question 13 : Taille d'un tenseur après upsampling avec ConvTranspose2d 2x2 sur un tensor 64x64.

Réponse :

Upsampling double la taille → 128x128.

Question 16 : Quel est le rôle des **skip connections** ?

Réponse : Transférer les détails fins de l'encodeur vers le décodeur pour améliorer la précision du masque.

Question 17 : Quelle est la taille d'un tenseur après deux blocs de pooling 2x2 si l'entrée est 256x256 ?

Réponse : 256 → 128 après 1er bloc → 64 après 2e bloc.

Question 18 : Que fait une **deconvolution / transpose convolution** ?

Réponse : Augmente la dimension spatiale pour reconstruire l'image (upsampling).

Question 19 : Quelle métrique utilise-t-on pour la segmentation binaire ?

Réponse : Le **Dice coefficient**, 1 = masque parfait.

Question 20 : Que se passe-t-il si on supprime le **bottleneck** dans U-Net ?

Réponse : Le réseau perd la capacité de capturer les informations globales profondes, segmentation moins précise.

♦ IV. Métriques et Évaluation

Question 21 : Qu'est-ce que le **Precision** et le **Recall** ?

Réponse : Precision = $TP/(TP+FP)$, Recall = $TP/(TP+FN)$. Mesurent respectivement la qualité et la complétude de la prédiction.

Question 22 : Qu'est-ce que le **F1 Score** ?

Réponse : Moyenne harmonique entre Precision et Recall : $F1 = 2 * (P*R)/(P+R)$.

Question 23 : Qu'est-ce que l'**IoU (Intersection over Union)** ?

Réponse : Surface d'intersection / surface d'union. Utilisée pour évaluer si la prédiction couvre correctement la vérité terrain.

Question 24 : Qu'est-ce que la **Mean Average Precision (mAP)** ?

Réponse : Moyenne des précisions sur tous les seuils de l'IoU, mesure globale de performance pour détection/segmentation.

Question 25 : Pourquoi la normalisation des images est importante avant d'entraîner AlexNet ou U-Net ?

Réponse : Permet d'accélérer la convergence et d'éviter que certaines valeurs de pixel dominent l'apprentissage.